Trabajo Práctico 1: Métodos de Búsqueda

Sistemas de Inteligencia Artificial

Guion de Presentación

ITBA

2026

Nota para el presentador

Este documento está pensado para ser leído en voz alta durante la presentación virtual. Cada sección corresponde a una parte de la exposición. Las notas en recuadros amarillos como este son indicaciones para vos y no se leen en voz alta.

Índice

1. Introducción **3**

2. Ejercicio 1: Heurísticas para el 8-puzzle **3**
2.1. El problema . . . 3
2.2. Observación clave: la ficha 5 . . . 3
2.3. Heurística h_5: forzar la ficha 5 al centro . . . 3
2.4. Heurística h_G: distancia Manhattan mínima sobre todos los goals . . . 4
2.5. Combinación final . . . 4

3. Ejercicio 2: Sokoban – Descripción del problema **4**
3.1. Qué es Sokoban . . . 4
3.2. Representación del estado . . . 5
3.3. BoardLayout: datos estáticos del tablero . . . 5

4. Algoritmos de búsqueda implementados **5**
4.1. BFS (Breadth-First Search) . . . 5
4.2. DFS (Depth-First Search) . . . 5
4.3. Greedy Search . . . 5
4.4. A* . . . 6

5. Heurística 1: Detección de deadlocks estáticos **6**
5.1. La idea . . . 6
5.2. El algoritmo: BFS inverso . . . 6
5.3. Formalización . . . 6
5.4. Admisibilidad . . . 7
5.5. Uso en la búsqueda . . . 7

6. Heurística 2: Minimum matching (algoritmo Húngaro) **7**
6.1. La idea . . . 7
6.2. Por qué no usar greedy nearest-neighbor . . . 7
6.3. Implementación . . . 8
6.4. Admisibilidad . . . 8

7. Combinación de heurísticas **8**

8. Resultados experimentales **9**
8.1. Costo por nivel . . . 9
8.2. Nivel 3 – Sala abierta (2 cajas, tablero abierto) . . . 9
8.3. Análisis de los resultados . . . 10
8.4. Gráficos de algoritmos óptimos . . . 10
8.5. Gráficos de algoritmos no óptimos . . . 11
8.6. Comparativas directas . . . 11

9. Arquitectura del código **11**

10. Tests **12**

11. Conclusiones **12**

1. Introducción

Buenas, voy a presentar el Trabajo Práctico 1 de Sistemas de Inteligencia Artificial, que trata sobre **métodos de búsqueda**.

El trabajo tiene dos ejercicios. El primero es un análisis teórico de heurísticas para el **8-puzzle con múltiples estados objetivo**. El segundo es la implementación completa de un solver de **Sokoban** usando algoritmos de búsqueda informados y desinformados, con al menos dos heurísticas admisibles.

Voy a empezar con el ejercicio teórico y después paso a la implementación.

2. Ejercicio 1: Heurísticas para el 8-puzzle

2.1. El problema

El 8-puzzle es un tablero de 3×3 con fichas numeradas del 1 al 8 y un espacio vacío. El objetivo es llegar a un estado meta deslizando fichas hacia el espacio vacío. En la variante que analizamos, **hay más de un estado objetivo posible**, lo cual complica el diseño de heurísticas porque no podemos calcular la distancia a un único estado final.

2.2. Observación clave: la ficha 5

Lo primero que notamos es una propiedad estructural del tablero. En una grilla 3×3, la **única casilla que tiene 4 vecinos es el centro**. Las esquinas tienen 2 vecinos y los bordes tienen 3. Esto significa que la ficha 5, que en el estado objetivo necesita estar adyacente a las fichas 4, 6, 2 y 8, **necesariamente tiene que terminar en el centro**. No hay otra posición que le permita tener 4 vecinos.

2.3. Heurística h_5: forzar la ficha 5 al centro

A partir de esa observación, diseñamos una heurística $h_5(s)$ que estima cuántos movimientos faltan para llevar la ficha 5 al centro. No es simplemente la distancia Manhattan de la ficha 5 al centro, porque hay que considerar dónde está el espacio vacío:

- Si la ficha 5 está en una **esquina** y el espacio vacío no está adyacente: $h_5 = 3$
- Si la ficha 5 está en una **esquina** y el espacio vacío está adyacente: $h_5 = 2$ (pero puede ser 4 en el peor caso)
- Si la ficha 5 está en un **borde** y el espacio vacío no está en el centro: $h_5 = 1$
- Si la ficha 5 está en un **borde** y el espacio vacío está en el centro: $h_5 = 2$
- Si la ficha 5 ya está en el **centro**: $h_5 = 0$

Esta heurística es **admisible** porque nunca sobreestima: en cada caso, el valor es menor o igual a la cantidad real de movimientos necesarios.

2.4. Heurística h_G: distancia Manhattan mínima sobre todos los goals

Para la segunda heurística, usamos la distancia Manhattan clásica pero adaptada a múltiples estados objetivo. La idea es:

$$h_G(s) = \min_{\text{goal } g \in G} \sum_{i=1}^{8} \text{Manhattan}(\text{pos}_s(i),\ \text{pos}_g(i))$$

Es decir, para cada estado objetivo posible, calculamos la suma de distancias Manhattan de cada ficha a su posición en ese goal, y nos quedamos con el **mínimo**. Esto es admisible porque estamos tomando el escenario más optimista.

2.5. Combinación final

La heurística final que proponemos es:

$$h(s) = \text{máx}\left(h_5(s),\ h_G(s)\right)$$

Nota para el presentador

Si te preguntan por qué usamos máx y no suma: explicar que la suma puede doblecontar esfuerzo compartido y romper admisibilidad. El máx de dos heurísticas admisibles es siempre admisible.

Usamos el **máximo** y no la suma. Esto es importante: si sumáramos ambas heurísticas, podríamos estar contando doble el esfuerzo de ciertos movimientos, lo cual rompería la admisibilidad. En cambio, el máximo de dos heurísticas admisibles es siempre admisible, y nos da una cota inferior más ajustada que cualquiera de las dos por separado.

3. Ejercicio 2: Sokoban – Descripción del problema

3.1. Qué es Sokoban

Sokoban es un puzzle donde un jugador tiene que empujar cajas hasta posiciones objetivo dentro de un tablero con paredes. Las reglas son simples pero generan una complejidad enorme:

- El jugador se mueve en 4 direcciones: arriba, abajo, izquierda, derecha.
- Las cajas solo se pueden **empujar**, no tirar.
- Solo se puede empujar **una caja a la vez**.
- El puzzle se resuelve cuando **todas las cajas están sobre todos los goals**.

Lo que hace a Sokoban particularmente difícil para la búsqueda es que es muy fácil llegar a estados irrecuperables. Si empujás una caja a una esquina que no es un goal, ya no podés sacarla de ahí. Esto hace que las heurísticas sean cruciales para guiar la búsqueda.

3.2. Representación del estado

Modelamos cada estado del juego con la clase `SokobanState`, que tiene:

- **Posición del jugador**: una tupla $(fila, columna)$.
- **Posiciones de las cajas**: un `frozenset` de posiciones. Usamos `frozenset` porque es inmutable y hasheable, lo que nos permite meter estados en un conjunto de explorados eficientemente.
- **Goals y paredes**: vienen del `BoardLayout`, que es la información estática del tablero.

La condición de goal es simplemente que el conjunto de posiciones de cajas sea igual al conjunto de posiciones de goals.

3.3. BoardLayout: datos estáticos del tablero

Separamos los datos estáticos del tablero en una clase `BoardLayout` que se comparte entre todos los estados de un mismo nivel. Esto contiene:

- Las paredes, los goals, y el piso (posiciones caminables).
- Se cachea con `@lru_cache` para no recalcular.

Esta separación es clave para la performance: en vez de copiar toda la información del tablero en cada estado, compartimos una referencia al mismo layout.

4. Algoritmos de búsqueda implementados

Implementamos cuatro algoritmos. Los dos primeros son **desinformados** y los dos últimos son **informados**:

4.1. BFS (Breadth-First Search)

BFS explora nivel por nivel usando una cola FIFO. Garantiza encontrar la solución de **costo óptimo** cuando todos los movimientos tienen costo uniforme, que es nuestro caso. El problema es que puede explorar una cantidad enorme de nodos.

4.2. DFS (Depth-First Search)

DFS explora en profundidad usando una pila LIFO. Es más eficiente en memoria pero **no garantiza optimalidad**. En Sokoban puede encontrar soluciones muy largas e innecesarias, como vamos a ver en los resultados.

4.3. Greedy Search

Greedy usa una cola de prioridad ordenada solo por el valor heurístico $h(n)$. Siempre expande el nodo que *parece* más prometedor. Es rápido pero **no garantiza optimalidad** porque ignora el costo acumulado.

4.4. A*

A* usa una cola de prioridad ordenada por $f(n) = g(n) + h(n)$, donde $g(n)$ es el costo acumulado y $h(n)$ es la heurística. Si la heurística es **admisible** (nunca sobreestima), A* **garantiza encontrar la solución óptima**. Es el algoritmo más inteligente de los cuatro: balancea entre el costo real y la estimación a futuro.

Nuestra implementación usa **graph search con reapertura de nodos**: si se encuentra un camino más barato a un estado ya expandido, se lo reabre y re-expande. Esto garantiza optimalidad con heurísticas admisibles, sin requerir consistencia.

Nota para el presentador

Si preguntan por IDDFS: decir que era opcional y no se implementó, pero explicar brevemente que combina la completitud de BFS con la eficiencia en memoria de DFS, usando profundizaciones iterativas.

5. Heurística 1: Detección de deadlocks estáticos

5.1. La idea

La primera heurística que implementamos se basa en detectar posiciones donde una caja **nunca puede llegar a ningún goal**, sin importar qué movimientos se hagan. Si una caja está en una de esas posiciones, el estado es irrecuperable y la heurística devuelve **infinito**.

El ejemplo más simple es una esquina: si empujás una caja a una esquina que no es un goal, no hay forma de sacarla. Pero hay casos más sutiles, como posiciones a lo largo de una pared donde no hay goals.

5.2. El algoritmo: BFS inverso

Para calcular qué posiciones son alcanzables para una caja, hacemos un **BFS inverso desde los goals**:

1. Partimos con todos los goals como frontera.
2. Para cada posición, calculamos de dónde podría haber venido una caja con un empuje válido. Es decir, si una caja está en la posición p, ¿desde qué posición q se la pudo haber empujado a p? Para eso, q y la posición de soporte del jugador (detrás de q) tienen que ser piso.
3. Todas las posiciones del piso que **no** fueron alcanzadas por este BFS son **posiciones prohibidas**.

Este análisis es **estático**: se calcula una sola vez por tablero y se cachea.

5.3. Formalización

$$h_{\text{deadlock}}(s) = \begin{cases} +\infty & \text{si alguna caja está en una posición prohibida} \\ 0 & \text{en caso contrario} \end{cases}$$

5.4. Admisibilidad

Esta heurística es admisible porque si devuelve infinito, el estado es verdaderamente irrecuperable: no existe ninguna secuencia de movimientos que lleve esa caja a un goal. Cuando devuelve 0, nunca sobreestima. Es una heurística binaria: o poda el estado completamente, o no aporta información numérica.

5.5. Uso en la búsqueda

Además de usarse como heurística, la detección de deadlocks se usa para **podar sucesores** en Greedy y A*. Si un movimiento empuja una caja a una posición prohibida, directamente no generamos ese estado hijo. Esto reduce significativamente el espacio de búsqueda.

6. Heurística 2: Minimum matching (algoritmo Húngaro)

6.1. La idea

La segunda heurística calcula el **costo mínimo de asignar cada caja a un goal**, usando la distancia Manhattan como costo de cada par. Es un problema de asignación uno-a-uno: cada caja se asigna a exactamente un goal, y queremos minimizar la suma total.

$$h_{\text{match}}(s) = \min_{\sigma} \sum_{i} \text{Manhattan}(\text{caja}_i,\ \text{goal}_{\sigma(i)})$$

donde σ recorre todas las asignaciones posibles (permutaciones).

6.2. Por qué no usar greedy nearest-neighbor

Nota para el presentador

Este punto es clave para la defensa. Si te preguntan "¿por qué no asignar cada caja al goal más cercano?", tenés un ejemplo concreto preparado.

Uno podría pensar en simplemente asignar cada caja al goal más cercano, pero eso puede dar un resultado subóptimo. Tenemos un test que lo demuestra con un ejemplo concreto:

Elemento	Posición
Caja A	$(2, 2)$
Caja B	$(6, 5)$
Goal X	$(2, 7)$
Goal Y	$(6, 2)$

Con **greedy nearest-neighbor**, procesando las cajas en orden:

- Caja A: su goal más cercano es Y en $(6, 2)$ a distancia $|2-6| + |2-2| = 4$.

- Caja B queda forzada a tomar X en $(2, 7)$: distancia $|6-2| + |5-7| = 6$.
- Total greedy: $4 + 6 = \mathbf{10}$.

Con el **algoritmo Húngaro** (asignación óptima):

- Caja A $\rightarrow$ Goal X: $|2-2| + |2-7| = 5$.
- Caja B $\rightarrow$ Goal Y: $|6-6| + |5-2| = 3$.
- Total óptimo: $5 + 3 = \mathbf{8}$.

El greedy da 10, el Húngaro da 8. La diferencia es que el greedy asigna la primera caja de forma localmente óptima pero globalmente subóptima: "se roba" un goal que le convenía más a la otra caja.

6.3. Implementación

Para resolver el problema de asignación usamos `scipy.optimize.linear_sum_assignment`, que implementa el algoritmo Húngaro en $O(n^3)$ donde n es la cantidad de cajas.

Hay dos optimizaciones para evitar overhead innecesario:

1. **Cajas ya en goals**: solo consideramos cajas que aún no están en un goal y goals que aún están vacíos. Las cajas ya resueltas contribuyen 0 al costo.
2. **Shortcircuit para una sola caja**: si queda una sola caja pendiente, retornamos la distancia Manhattan directamente sin instanciar NumPy. Esto es relevante porque A* visita muchos estados casi resueltos.

6.4. Admisibilidad

La heurística es **admisible** porque:

- La distancia Manhattan es un lower bound del costo real de empujar una caja a un goal (ignora paredes y otras cajas).
- El matching mínimo no sobreestima porque asigna cada goal a una sola caja (sin doble conteo).
- Es decir, $h_{\text{match}}(s) \leq h^*(s)$ para todo estado s.

7. Combinación de heurísticas

Combinamos ambas heurísticas tomando el **máximo**:

$$h_{\text{combined}}(s) = \text{máx}\left(h_{\text{match}}(s),\ h_{\text{deadlock}}(s)\right)$$

¿Por qué el máximo y no la suma?

- La **suma** podría doble-contar el esfuerzo: ambas heurísticas estiman aspectos del mismo costo real, y sumarlas podría sobreestimar, rompiendo la admisibilidad.

- El **máximo** de dos heurísticas admisibles es siempre admisible: $\text{máx}(h_1, h_2) \leq \text{máx}(h^*, h^*) = h^*$.
- En la práctica, h_{deadlock} aporta la poda de estados imposibles (devuelve infinito) y h_{match} aporta la guía numérica (cuánto falta). Se complementan perfectamente.

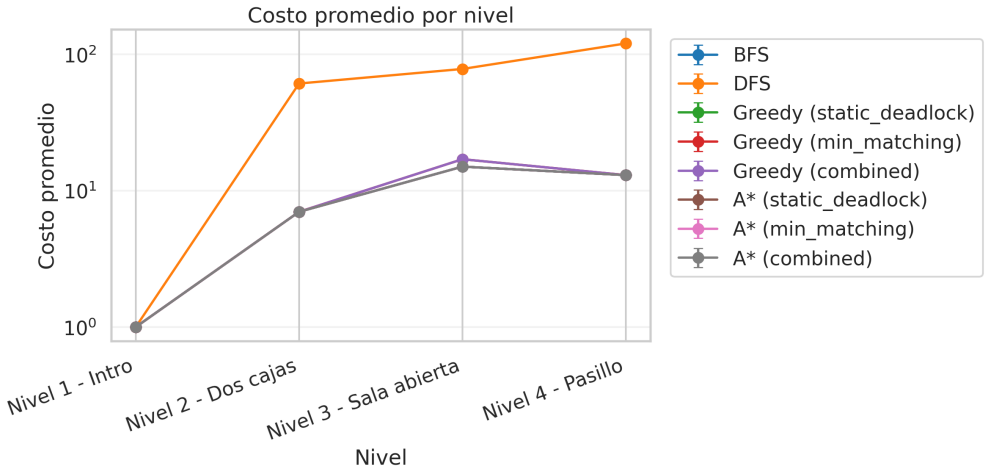
8. Resultados experimentales

Nota para el presentador

Estos son los resultados reales de correr el solver. Si podés, mostrá la terminal con el output de `python3 -m src.main` en simultáneo.

Corrimos los 8 métodos sobre 4 niveles con 10 iteraciones (seed=42). Todos resolvieron todos los niveles. Voy a mostrar los resultados más representativos.

8.1. Costo por nivel



BFS y A* encuentran la solución óptima en todos los niveles. DFS da costos muy superiores: 61, 78 y 120 donde el óptimo es 7, 15 y 13.

8.2. Nivel 3 – Sala abierta (2 cajas, tablero abierto)

Este es el nivel que mejor muestra las diferencias, porque el branching alto amplifica el efecto de las heurísticas:

Método	Heurística	Costo	Nodos exp.	Frontera	Tiempo
BFS	ninguna	15	4 171	879	26 ms
DFS	ninguna	78	3 179	75	15 ms
Greedy	static_deadlock	15	680	207	5 ms
Greedy	min_matching	17	47	31	1 ms
Greedy	combined	17	47	31	1 ms
A*	static_deadlock	15	680	125	5 ms
A*	min_matching	15	133	91	3 ms
A*	combined	15	133	91	3 ms

8.3. Análisis de los resultados

DFS encuentra soluciones mucho peores. En Nivel 3, DFS da costo 78 cuando el óptimo es 15. En Nivel 4 da 120 vs 13. Esto es esperable: DFS no tiene noción de costo.

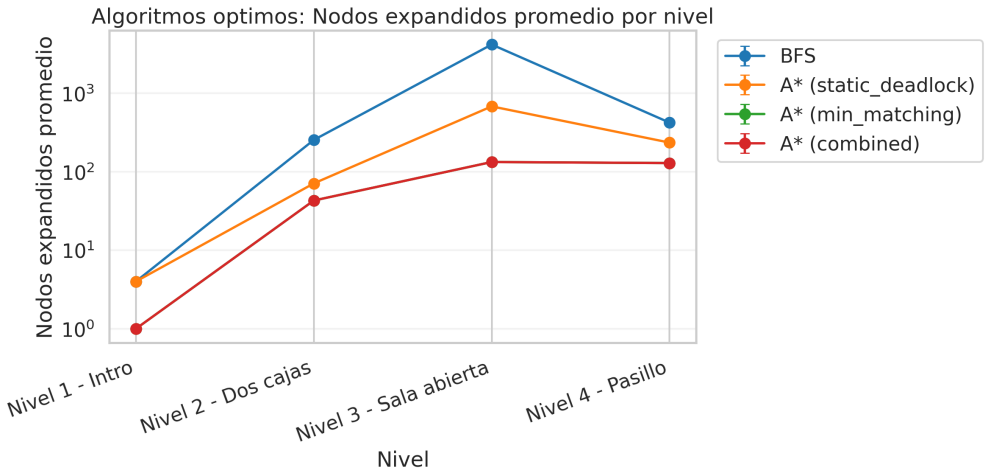
BFS encuentra el óptimo pero explora más. BFS expande 4 171 nodos en Nivel 3 para encontrar costo 15. Es correcto pero costoso.

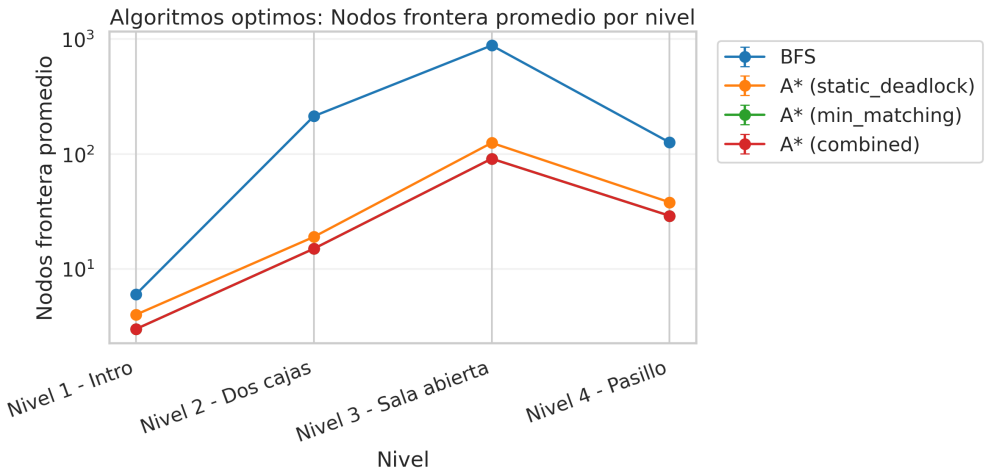
static_deadlock sola no guía la búsqueda. A* con static_deadlock expande los mismos 680 nodos que Greedy con static_deadlock, porque $h = 0$ para estados no muertos. Solo aporta poda binaria (0 o ∞), sin información numérica.

min_matching es la diferencia clave. A* con min_matching baja de 680 a **133 nodos** — 5× menos que con static_deadlock y 31× menos que BFS. La heurística da información precisa sobre cuánto falta.

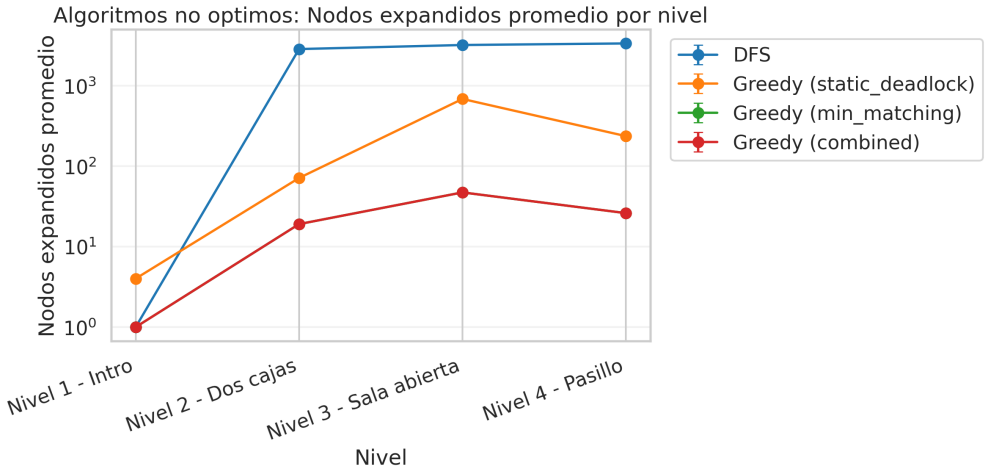
combined no mejora sobre min_matching. En estos niveles, máx(matching, deadlock) da los mismos nodos que matching solo. min_matching ya es consistente, así que A* no necesita reabrir nodos.

8.4. Gráficos de algoritmos óptimos

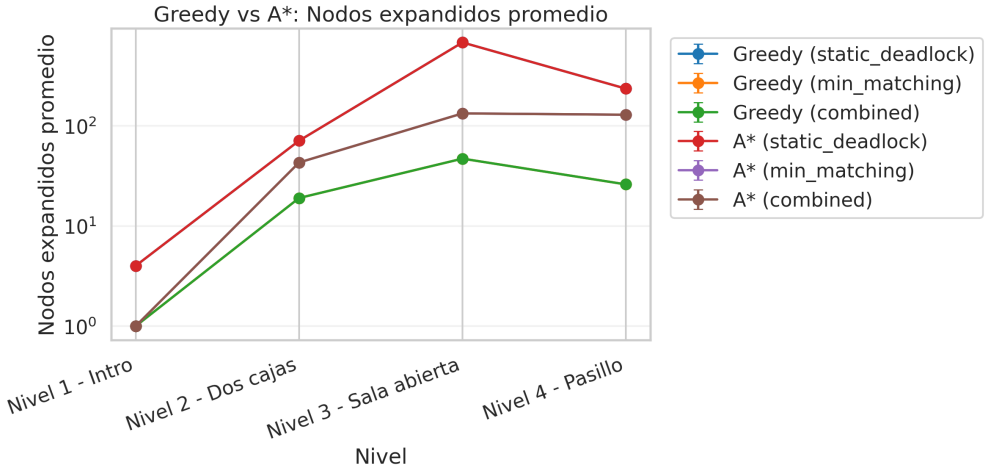




8.5. Gráficos de algoritmos no óptimos



8.6. Comparativas directas



Greedy expande menos nodos (no busca optimalidad) pero A* garantiza costo óptimo. A*(static_deadlock) expande mucho más porque $h = 0$ no guía.

9. Arquitectura del código

Nota para el presentador

Esta sección es útil si te piden que expliques la estructura del código o si querés mostrar archivos específicos.

El proyecto está organizado en módulos:

- `src/model/board_layout.py`: datos estáticos del tablero (paredes, goals, piso). Se comparte entre estados.
- `src/model/state.py`: la clase `SokobanState` con la lógica de movimientos y generación de sucesores.

- `src/engine/search.py`: los cuatro algoritmos de búsqueda. Cada uno usa la estructura de datos apropiada (cola, pila, o heap).
- `src/heuristics/static_deadlocks.py`: BFS inverso para detección de deadlocks estáticos.
- `src/heuristics/min_matching.py`: asignación mínima con el algoritmo Húngaro.
- `src/heuristics/sokoban_heuristics.py`: registro de heurísticas, combinación con `max`, y resolución por nombre.

Decisiones de diseño clave:

1. **Inmutabilidad**: usamos `frozenset` para cajas y goals, lo que permite hashear estados y meterlos en el conjunto de explorados.
2. **Cache**: el análisis de deadlocks se computa una vez por layout con `@lru_cache`.
3. **Separación layout/estado**: el `BoardLayout` se comparte entre todos los estados, evitando copiar paredes y goals en cada movimiento.

10. Tests

Tenemos 36 tests unitarios en 7 archivos:

- **Tests de A*** (5 tests): reapertura de nodos cerrados, entradas stale en el heap, tie-breaking determinista, cache de heurística, y que A* iguala costo de BFS.
- **Tests de algoritmos** (6 tests): DFS encuentra solución, Greedy encuentra solución, Greedy $\geq$ BFS en costo, estado ya resuelto retorna costo 0, roundtrip de render, guard de recursión en combined.
- **Tests de min_matching** (9 tests): cajas en goals, shortcircuit una caja, matching óptimo vs greedy, admisibilidad, combinada, A* iguala BFS con min_matching y combined.
- **Tests de deadlocks** (8 tests): goals no prohibidos, esquinas prohibidas, BFS inverso, poda de sucesores, heurística infinito, combinada, cache por layout.
- **Tests de IO, pipeline y CLI** (8 tests): parser de niveles, runner de benchmarks, policy de deadlocks.

Los 36 tests pasan correctamente.

11. Conclusiones

Para cerrar, las conclusiones principales del trabajo:

1. **Las heurísticas informadas reducen drásticamente el espacio explorado.** Pasamos de decenas de miles de nodos con BFS a decenas o cientos con heurísticas informadas. Esto es lo que hace viable resolver puzzles más grandes.

2. **No todas las heurísticas son iguales.** La detección de deadlocks es binaria: poda estados imposibles pero no dice cuánto falta. El minimum matching da información numérica que guía la búsqueda mucho más efectivamente.

3. **La combinación con max es la forma segura.** Combinar heurísticas con el máximo mantiene la admisibilidad y aprovecha lo mejor de cada una.

4. **A* con heurística admisible es el mejor compromiso.** Garantiza optimalidad y, con una buena heurística, explora una fracción del espacio total.

5. **El matching óptimo no es trivial.** Como mostramos con el ejemplo concreto, la asignación greedy puede ser subóptima. El algoritmo Húngaro resuelve esto en $O(n^3)$, lo cual es despreciable para la cantidad de cajas típica en Sokoban.

Eso es todo. Quedo a disposición para preguntas.

Apéndice: Preguntas frecuentes de defensa

Nota para el presentador

Estas son respuestas preparadas para preguntas típicas que pueden hacer los profesores. No las leés en la presentación, las tenés de backup.

¿Por qué Manhattan y no distancia real? Porque la distancia Manhattan es un lower bound de la distancia real (ignora paredes). Eso la hace admisible. Calcular la distancia real requeriría un BFS por cada par caja-goal, lo cual es más costoso computacionalmente.

¿Qué pasa si hay más cajas que goals o viceversa? En Sokoban estándar siempre coinciden. Nuestro `is_goal()` chequea que los conjuntos sean iguales. El matching solo considera cajas fuera de goals y goals vacíos, que siempre tienen la misma cantidad.

¿La heurística de deadlocks captura todos los deadlocks? No, solo los estáticos. Existen deadlocks dinámicos (como dos cajas trabadas entre sí contra una pared) que requieren un análisis más complejo y no los implementamos. Nuestro análisis es *sound* pero no completo: todo lo que marca como deadlock realmente lo es, pero puede haber deadlocks que no detecta.

¿Por qué no implementaron IDDFS? Era opcional. IDDFS combina la completitud de BFS con la eficiencia en memoria de DFS usando profundizaciones iterativas. Es útil cuando la memoria es limitada, pero en nuestros niveles de prueba A* con buenas heurísticas es superior.

¿Qué pasa con niveles más grandes? En niveles con muchas cajas, el espacio de estados crece exponencialmente. Las heurísticas ayudan a podar pero no eliminan la complejidad inherente. Sokoban es PSPACE-complete, así que no hay algoritmo eficiente en el caso general.

¿Por qué scipy y no una implementación propia del Húngaro? `scipy.optimize.linear_sum` es una implementación optimizada en C del algoritmo Húngaro. Reimplementarlo en Python puro sería más lento y propenso a bugs. En un contexto académico, es razonable usar bibliotecas establecidas para problemas bien conocidos.